Laporan Tugas Sesi 8

Unit Testing dengan JUnit 5 pada Class `OrderPricing`

Nama	Bambang Herlambang (NRP 5803024019)
Mata kuliah	Software Testing, Universitas Katolik Widya Mandala Surabaya
Dosen pengampu	Philipus Suryo Subandoro
Tanggal	19 Oktober 2026
Repositori	`https://github.com/Bambang-code/software-testing-sesi8-junit`

1 Pendahuluan

1.1 Latar Belakang dan Tujuan

Unit testing menguji bagian terkecil kode yang dapat diuji, yaitu method atau class, secara terisolasi. Pada piramida pengujian, unit test berada di dasar karena jumlahnya paling banyak, paling cepat dijalankan, dan paling murah dipelihara. Tugas Sesi 8 meminta pembuatan satu class Java dengan minimal tiga method berlogika bersyarat, lalu pengujiannya dengan JUnit 5. Laporan ini mendokumentasikan rancangan, implementasi, dan hasil eksekusi pengujian tersebut.

1.2 Ketentuan Tugas dan Pemenuhannya

No	Ketentuan	Pemenuhan
1	Minimal 8 test, termasuk 1 `@ParameterizedTest` berbasis BVA	16 method test (31 kasus uji), 3 di antaranya parameterized berbasis BVA
2	Minimal 2 test `assertThrows` untuk skenario negatif	4 method test dengan 4 pemanggilan `assertThrows`, semua memverifikasi pesan exception
3	Penggunaan `@BeforeEach` dan `@DisplayName`	`setUp()` dengan `@BeforeEach`; semua test memakai `@DisplayName`
4	Tangkapan layar hasil `mvn test` yang lulus seluruhnya	Gambar 1: 31 test lulus, `BUILD SUCCESS`

2 Class yang Diuji

Class `OrderPricing` (paket `marketplace`) berisi logika harga pesanan pada marketplace UMKM lokal. Tema ini dipilih agar sejalan dengan topik sistem yang dikerjakan pada proyek lintas mata kuliah. Class memiliki tiga method berlogika bersyarat.

Method	Aturan	Input tidak valid
`hitungTotalSetelahDiskon(subtotal)`	Subtotal di bawah 100.000: tanpa diskon. Mulai 100.000 sampai di bawah 500.000: diskon 5%. Mulai 500.000: diskon 10%.	Subtotal negatif
`hitungOngkir(beratKg, ekspres)`	Sampai 1 kg: Rp10.000. Di atas 1 sampai 5 kg: Rp20.000. Di atas 5 kg: Rp20.000 ditambah Rp5.000 per kg tambahan (dibulatkan ke atas). Ekspres dikenai tarif 1,5 kali.	Berat nol atau negatif
`klasifikasiPelanggan(jumlahPesanan)`	0 sampai 4 pesanan: BARU. 5 sampai 19: REGULER. 20 atau lebih: LOYAL.	Jumlah pesanan negatif

Input tidak valid ditolak dengan `IllegalArgumentException`. Kode lengkap tersedia pada Lampiran A.

3 Rancangan Pengujian

3.1 Teknik Penentuan Data Uji

Data uji ditentukan dengan Equivalence Partitioning (EP) dan Boundary Value Analysis (BVA). EP membagi input menjadi kelompok yang diperlakukan sama oleh program (misalnya subtotal tanpa diskon, diskon 5%, diskon 10%, dan tidak valid). BVA kemudian memilih nilai di sekitar batas antar kelompok, yaitu tepat di batas, satu langkah di bawah, dan satu langkah di atas, karena kesalahan pada kondisi seperti `>` versus `>=` paling sering muncul di titik tersebut.

3.2 Nilai Uji dan Hasil yang Diharapkan

Method	Nilai uji	Hasil yang diharapkan
`hitungTotal SetelahDiskon`	0; 99.999; 100.000; 100.001; 499.999; 500.000; 500.001	0; 99.999; 95.000; 95.000,95; 474.999,05; 450.000; 450.000,90
	−1	`IllegalArgumentException` (“Subtotal negatif”)
`hitungOngkir`	0,01 kg; 0,99 kg; 1 kg; 1,01 kg; 4,99 kg; 5 kg; 5,01 kg; 5,1 kg; 7 kg (reguler)	Rp10.000; Rp10.000; Rp10.000; Rp20.000; Rp20.000; Rp20.000; Rp25.000; Rp25.000; Rp30.000
	1 kg (ekspres)	Rp15.000
	0 kg; −2,5 kg	`IllegalArgumentException` (“Berat harus lebih dari 0”)
`klasifikasi Pelanggan`	0; 4; 5; 6; 19; 20; 21	BARU; BARU; REGULER; REGULER; REGULER; LOYAL; LOYAL
	−1	`IllegalArgumentException` (“Jumlah pesanan negatif”)

Pengujian skenario negatif disertakan pada setiap method, karena test yang hanya melewati jalur sukses memberi rasa aman palsu.

4 Implementasi Test

4.1 Anotasi dan Assertion yang Digunakan

Elemen	Penggunaan pada test class
`@Test`	Menandai method sebagai test biasa.
`@BeforeEach`	`setUp()` membuat objek `OrderPricing` baru sebelum tiap test (tahap Arrange), sehingga tidak ada keadaan yang terbawa antar test.
`@DisplayName`	Nama test yang mudah dibaca pada laporan, ada di semua test.
`@ParameterizedTest` dan `@CsvSource`	Satu logika test dijalankan dengan tabel data hasil BVA (7 kasus untuk diskon, 4 kasus untuk ongkir, 7 kasus untuk klasifikasi).
`assertEquals`	Membandingkan hasil dengan nilai harapan. Untuk tipe `double` dipakai toleransi (delta) 0,001.
`assertThrows`	Memastikan input tidak valid melempar `IllegalArgumentException`; semua test exception memverifikasi pesan galat dengan `assertEquals` pada `getMessage()`.

Setiap test mengikuti pola Arrange-Act-Assert: objek disiapkan oleh `setUp()`, method yang

diuji dipanggil (Act), lalu hasilnya diperiksa (Assert).

4.2 Daftar Test

No	Method test	Fokus	Kasus
1	`tanpaDiskonDiBawahBatas`	Batas: 99.999, tanpa diskon	1
2	`diskonLimaPersenDiBatasBawah`	Batas: 100.000, diskon 5%	1
3	`diskonSepuluhPersenDiBatasAtas`	Batas: 500.000, diskon 10%	1
4	`diskonNilaiBatas`	`@ParameterizedTest` BVA diskon	7
5	`subtotalNegatifDitolak`	Negatif: `assertThrows` dan pesan galat	1
6	`ongkirTepatDiBatasSatuKilo`	BVA: 1 kg (batas atas tier 1)	1
7	`ongkirTepatDiAtasSatuKilo`	BVA: 1,01 kg (tepat di atas batas tier 1)	1
8	`ongkirTepatDiBatasLimaKilo`	BVA: 5 kg (batas atas tier 2)	1
9	`ongkirTepatDiAtasLimaKilo`	BVA: 5,1 kg (tepat di atas batas tier 2)	1
10	`ongkirNilaiBatasTambahan`	`@ParameterizedTest` BVA ongkir: 0,01; 0,99; 4,99; 5,01 kg	4
11	`ongkirTierTigaVerifikasiFormula`	Verifikasi formula tier 3: 7 kg	1
12	`ongkirEkspres`	Tarif ekspres 1,5 kali	1
13	`beratNolDitolak`	Negatif: `assertThrows` dan pesan galat	1
14	`beratNegatifDitolak`	Negatif: `assertThrows` dan pesan galat	1
15	`klasifikasiNilaiBatas`	`@ParameterizedTest` BVA klasifikasi	7
16	`jumlahPesananNegatifDitolak`	Negatif: `assertThrows` dan pesan galat	1
	Total		**31**

Kode lengkap test class tersedia pada Lampiran B.

5 Hasil Eksekusi

5.1 Lingkungan

Komponen	Versi
Sistem operasi	Windows 11 (amd64)
JDK	Eclipse Temurin (Adoptium) 21.0.9
Target kompilasi	Java 17 (`maven.compiler.release`)
Build tool	Apache Maven 3.10.0
Plugin test	maven-surefire-plugin 3.2.5
Framework test	JUnit Jupiter 5.10.2 (`junit-jupiter`, mencakup modul `params`)
Editor	Visual Studio Code

5.2 Hasil mvn test

Pengujian dijalankan di PowerShell dari folder proyek yang berisi `pom.xml`. Karena Maven belum terdaftar permanen pada `PATH`, direktori `bin` Maven ditambahkan terlebih dahulu untuk sesi terminal tersebut, lalu `mvn test` dijalankan:

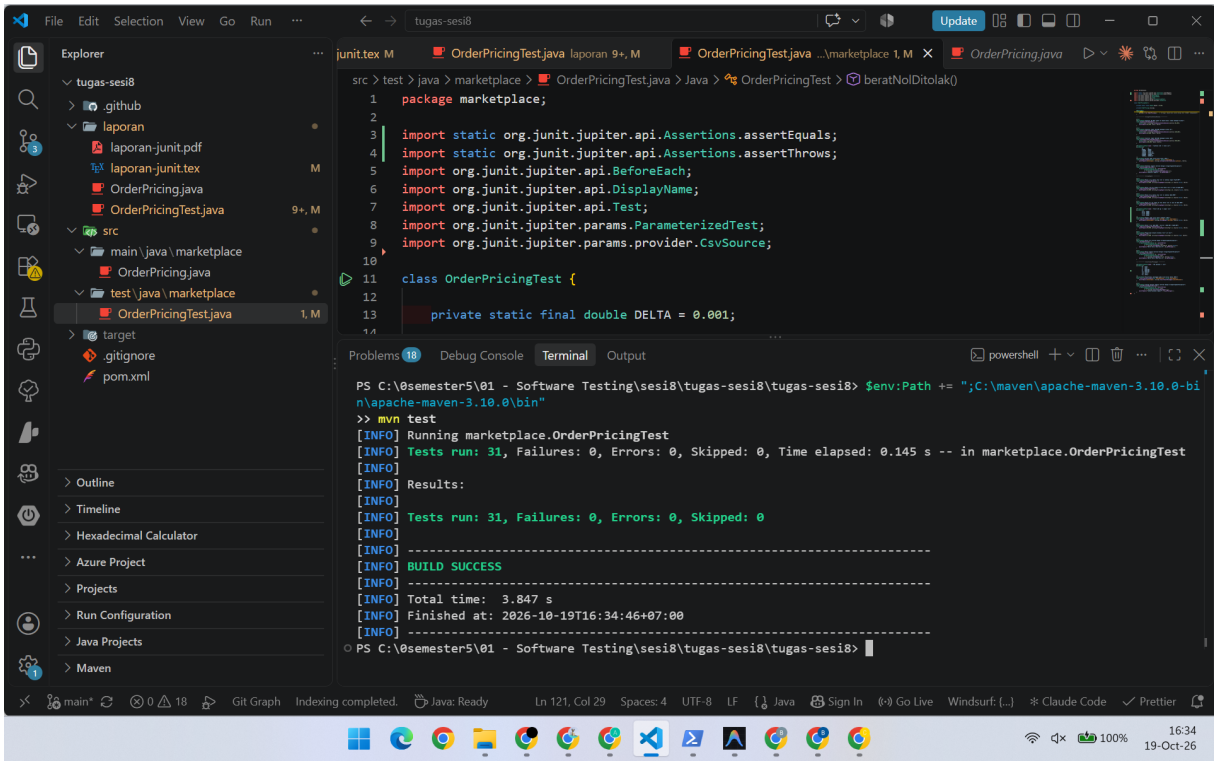
```
$env:Path += ";C:\maven\apache-maven-3.10.0-bin\apache-maven-3.10.0\bin"
mvn test
```

Hasilnya ditunjukkan pada Gambar 1.

```
PS C:\@semester5\01 - Software Testing\sesi8\tugas-sesi8\tugas-sesi8> $env:Path += ";C:\maven\apache-maven-3.10.0-bi
n\apache-maven-3.10.0\bin"
>> mvn test
[INFO] Running marketplace.OrderPricingTest
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.145 s -- in marketplace.OrderPricingTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.847 s
[INFO] Finished at: 2026-10-19T16:34:46+07:00
[INFO] ------------------------------------------------------------------------
PS C:\@semester5\01 - Software Testing\sesi8\tugas-sesi8\tugas-sesi8>
```

Gambar 1: Keluaran terminal `mvn test`: `Tests run: 31, Failures: 0, Errors: 0, Skipped: 0` dan `BUILD SUCCESS`.

Gambar 2 menampilkan tangkapan layar utuh pada saat yang sama, termasuk struktur proyek (`src/main`, `src/test`, `pom.xml`) dan status editor tanpa galat kompilasi (0 error).



Gambar 2: Tangkapan layar utuh Visual Studio Code setelah `mvn test` berhasil.

Indikator	Nilai
Tests run	31
Failures	0
Errors	0
Skipped	0
Waktu eksekusi test	0,145 detik
Status build	BUILD SUCCESS
Total waktu build	3,847 detik
Selesai pada	19 Oktober 2026, 16:34:46 (UTC+7)

Angka 31 berasal dari 13 test biasa ditambah 18 kasus pada tiga test parameterized (7 kasus diskon, 4 kasus ongkir, dan 7 kasus klasifikasi). Seluruhnya lulus, dan tidak ada test yang dilewati.

6 Evaluasi terhadap Kriteria Penilaian

6.1 Kriteria Tugas

Kriteria	Bobot	Bukti pada tugas ini
Ketepatan anotasi dan assertion	30%	Anotasi siklus hidup, parameterized, dan `@DisplayName` dipakai sesuai fungsinya; `assertEquals` memakai delta untuk `double`, `assertThrows` untuk exception dengan verifikasi pesan galat pada semua test negatif.
Cakupan skenario (batas dan negatif)	35%	Nilai batas dari BVA pada seluruh batas antarkelas di tiga method (tepat di bawah, tepat pada, dan tepat di atas batas), serta skenario negatif untuk setiap method.
Kualitas penamaan dan kepatuhan FIRST	20%	Nama method dan `@DisplayName` deskriptif; prinsip FIRST dibahas pada subbab berikut.
Bukti eksekusi	15%	Gambar 1.

6.2 Kepatuhan terhadap Prinsip FIRST

Prinsip	Penerapan
Fast	Seluruh 31 kasus selesai dalam 0,145 detik karena tidak ada akses database, jaringan, atau berkas.
Independent	`@BeforeEach` membuat objek baru untuk tiap test, tidak ada variabel statis yang berubah, sehingga urutan eksekusi tidak berpengaruh.
Repeatable	Tidak bergantung pada waktu, data acak, atau lingkungan eksternal; hasil sama di komputer mana pun.
Self-validating	Setiap test memakai assertion sehingga lulus atau gagal otomatis tanpa pemeriksaan manual.
Timely	Test ditulis bersamaan dengan class yang diuji, dengan data uji ditentukan dari aturan sebelum eksekusi.

6.3 Cakupan Cabang dan Keterbatasan

Penelusuran manual terhadap kode menunjukkan setiap cabang kondisi pada ketiga method dilalui oleh minimal satu test, termasuk cabang ekspres dan non-ekspres serta seluruh cabang validasi. Penelusuran ini bukan pengukuran dengan alat cakupan, karena proyek belum memakai alat seperti JaCoCo. Cakupan yang tinggi juga tidak menjamin tidak ada bug; nilai pengujian terletak pada kebermaknaan assertion dan pemilihan data uji.

7 Kesimpulan

Class `OrderPricing` dengan tiga method berlogika bersyarat telah diuji oleh 16 method test (31 kasus uji) memakai JUnit 5. Data uji ditentukan dengan BVA pada seluruh batas antarkelas di ketiga method (tepat di bawah, tepat pada, dan tepat di atas batas), dan dilengkapi skenario negatif untuk tiap method dengan verifikasi pesan exception. Seluruh test lulus pada eksekusi `mvn test` (31 dijalankan, 0 gagal, 0 error, 0 dilewati), dan rancangan test mengikuti pola Arrange-Act-Assert serta prinsip FIRST. Seluruh ketentuan Tugas Sesi 8 telah terpenuhi.

Lampiran A. Kode OrderPricing.java

```java
package marketplace;

/**
 * Logika harga pesanan pada marketplace UMKM lokal.
 * Berisi tiga method berlogika bersyarat: diskon, ongkir, dan klasifikasi pelanggan.
 */
public class OrderPricing {

    public static final double BATAS_DISKON_5 = 100_000;
    public static final double BATAS_DISKON_10 = 500_000;

    /**
     * Menghitung total setelah diskon.
     * Aturan: subtotal < 100.000 tanpa diskon, 100.000 s.d. < 500.000 diskon 5%,
     * >= 500.000 diskon 10%.
     *
     * @throws IllegalArgumentException jika subtotal negatif
     */
    public double hitungTotalSetelahDiskon(double subtotal) {
        if (subtotal < 0) {
            throw new IllegalArgumentException("Subtotal negatif");
        }
        if (subtotal >= BATAS_DISKON_10) {
            return subtotal * 0.90;
        }
        if (subtotal >= BATAS_DISKON_5) {
            return subtotal * 0.95;
        }
        return subtotal;
    }

    /**
     * Menghitung ongkos kirim berdasarkan berat (kg).
     * Aturan: <= 1 kg Rp10.000, > 1 s.d. 5 kg Rp20.000,
     * di atas 5 kg Rp20.000 + Rp5.000 per kg tambahan (dibulatkan ke atas).
     * Pengiriman ekspres dikenai tarif 1,5 kali.
     *
     * @throws IllegalArgumentException jika berat nol atau negatif
     */
    public double hitungOngkir(double beratKg, boolean ekspres) {
        if (beratKg <= 0) {
            throw new IllegalArgumentException("Berat harus lebih dari 0");
        }
        double ongkir;
        if (beratKg <= 1) {
            ongkir = 10_000;
        } else if (beratKg <= 5) {
            ongkir = 20_000;
        } else {
            ongkir = 20_000 + Math.ceil(beratKg - 5) * 5_000;
        }
        return ekspres ? ongkir * 1.5 : ongkir;
    }

    /**
```

```
56      * Mengklasifikasikan pelanggan berdasarkan jumlah pesanan.
57      * Aturan: 0-4 BARU, 5-19 REGULER, >= 20 LOYAL.
58      *
59      * @throws IllegalArgumentException jika jumlah pesanan negatif
60      */
61     public String klasifikasiPelanggan(int jumlahPesanan) {
62         if (jumlahPesanan < 0) {
63             throw new IllegalArgumentException("Jumlah pesanan negatif");
64         }
65         if (jumlahPesanan >= 20) {
66             return "LOYAL";
67         }
68         if (jumlahPesanan >= 5) {
69             return "REGULER";
70         }
71         return "BARU";
72     }
73 }
```

Lampiran B. Kode OrderPricingTest.java

```
1  package marketplace;
2
3  import org.junit.jupiter.api.BeforeEach;
4  import org.junit.jupiter.api.DisplayName;
5  import org.junit.jupiter.api.Test;
6  import org.junit.jupiter.params.ParameterizedTest;
7  import org.junit.jupiter.params.provider.CsvSource;
8
9  import static org.junit.jupiter.api.Assertions.*;
10
11 class OrderPricingTest {
12
13     private static final double DELTA = 0.001;
14
15     private OrderPricing pricing;
16
17     @BeforeEach
18     void setUp() {
19         pricing = new OrderPricing();   // Arrange: objek baru untuk setiap test (FIRST:
       Independent)
20     }
21
22     // ---------- hitungTotalSetelahDiskon ----------
23
24     @Test
25     @DisplayName("Subtotal 99.999 (tepat di bawah batas) tidak mendapat diskon")
26     void tanpaDiskonDiBawahBatas() {
27         double hasil = pricing.hitungTotalSetelahDiskon(99_999);
28         assertEquals(99_999, hasil, DELTA);
29     }
30
31     @Test
32     @DisplayName("Subtotal tepat 100.000 mendapat diskon 5%")
33     void diskonLimaPersenDiBatasBawah() {
34         double hasil = pricing.hitungTotalSetelahDiskon(100_000);
35         assertEquals(95_000, hasil, DELTA);
36     }
37
38     @Test
39     @DisplayName("Subtotal tepat 500.000 mendapat diskon 10%")
40     void diskonSepuluhPersenDiBatasAtas() {
41         double hasil = pricing.hitungTotalSetelahDiskon(500_000);
42         assertEquals(450_000, hasil, DELTA);
43     }
44
45     @ParameterizedTest(name = "subtotal {0} -> total {1}")
46     @CsvSource({
47             "0,       0",
48             "99999,   99999",
49             "100000,  95000",
50             "100001,  95000.95",
51             "499999,  474999.05",
52             "500000,  450000",
53             "500001,  450000.9"
54     })
```

```
55      @DisplayName("Diskon pada nilai-nilai batas (BVA)")
56      void diskonNilaiBatas(double subtotal, double diharapkan) {
57          assertEquals(diharapkan, pricing.hitungTotalSetelahDiskon(subtotal), DELTA);
58      }
59
60      @Test
61      @DisplayName("Subtotal negatif ditolak dengan IllegalArgumentException")
62      void subtotalNegatifDitolak() {
63          IllegalArgumentException ex = assertThrows(
64                  IllegalArgumentException.class,
65                  () -> pricing.hitungTotalSetelahDiskon(-1));
66          assertEquals("Subtotal negatif", ex.getMessage());
67      }
68
69      // ---------- hitungOngkir ----------
70
71      @Test
72      @DisplayName("Berat 1 kg (batas atas tier 1) dikenai ongkir Rp10.000")
73      void ongkirTepatDiBatasSatuKilo() {
74          assertEquals(10_000, pricing.hitungOngkir(1.0, false), DELTA);
75      }
76
77      @Test
78      @DisplayName("Berat 1,01 kg (tepat di atas batas tier 1) naik ke Rp20.000")
79      void ongkirTepatDiAtasSatuKilo() {
80          assertEquals(20_000, pricing.hitungOngkir(1.01, false), DELTA);
81      }
82
83      @Test
84      @DisplayName("Berat 5 kg (batas atas tier 2) dikenai Rp20.000")
85      void ongkirTepatDiBatasLimaKilo() {
86          assertEquals(20_000, pricing.hitungOngkir(5.0, false), DELTA);
87      }
88
89      @Test
90      @DisplayName("Berat 5,1 kg (tepat di atas batas tier 2) naik ke Rp25.000")
91      void ongkirTepatDiAtasLimaKilo() {
92          assertEquals(25_000, pricing.hitungOngkir(5.1, false), DELTA);
93      }
94
95      @ParameterizedTest(name = "berat {0} kg -> ongkir {1}")
96      @CsvSource({
97              "0.01, 10000",
98              "0.99, 10000",
99              "4.99, 20000",
100             "5.01, 25000"
101     })
102     @DisplayName("Ongkir pada nilai batas tambahan (BVA)")
103     void ongkirNilaiBatasTambahan(double beratKg, double diharapkan) {
104         assertEquals(diharapkan, pricing.hitungOngkir(beratKg, false), DELTA);
105     }
106
107     @Test
108     @DisplayName("Berat 7 kg: Rp20.000 + ceil(2) × Rp5.000 = Rp30.000")
109     void ongkirTierTigaVerifikasiFormula() {
110         assertEquals(30_000, pricing.hitungOngkir(7.0, false), DELTA);
111     }
```

```
112
113     @Test
114     @DisplayName("Pengiriman ekspres dikenai tarif 1,5 kali")
115     void ongkirEkspres() {
116         assertEquals(15_000, pricing.hitungOngkir(1.0, true), DELTA);
117     }
118
119     @Test
120     @DisplayName("Berat nol ditolak dengan IllegalArgumentException")
121     void beratNolDitolak() {
122         IllegalArgumentException ex = assertThrows(
123                 IllegalArgumentException.class,
124                 () -> pricing.hitungOngkir(0, false));
125         assertEquals("Berat harus lebih dari 0", ex.getMessage());
126     }
127
128     @Test
129     @DisplayName("Berat negatif ditolak dengan IllegalArgumentException")
130     void beratNegatifDitolak() {
131         IllegalArgumentException ex = assertThrows(
132                 IllegalArgumentException.class,
133                 () -> pricing.hitungOngkir(-2.5, true));
134         assertEquals("Berat harus lebih dari 0", ex.getMessage());
135     }
136
137     // ---------- klasifikasiPelanggan ----------
138
139     @ParameterizedTest(name = "{0} pesanan -> {1}")
140     @CsvSource({
141             "0,  BARU",
142             "4,  BARU",
143             "5,  REGULER",
144             "6,  REGULER",
145             "19, REGULER",
146             "20, LOYAL",
147             "21, LOYAL"
148     })
149     @DisplayName("Klasifikasi pelanggan pada nilai-nilai batas (BVA)")
150     void klasifikasiNilaiBatas(int jumlahPesanan, String diharapkan) {
151         assertEquals(diharapkan, pricing.klasifikasiPelanggan(jumlahPesanan));
152     }
153
154     @Test
155     @DisplayName("Jumlah pesanan negatif ditolak dengan IllegalArgumentException")
156     void jumlahPesananNegatifDitolak() {
157         IllegalArgumentException ex = assertThrows(
158                 IllegalArgumentException.class,
159                 () -> pricing.klasifikasiPelanggan(-1));
160         assertEquals("Jumlah pesanan negatif", ex.getMessage());
161     }
162 }
```

Daftar Pustaka

[1] JUnit Team. *JUnit 5 User Guide*. https://junit.org/junit5/docs/current/user-guide.

[2] Tahchiev, P., et al. *JUnit in Action*. Manning.

[3] Black, R., Graham, D., dan van Veenendaal, E. *Foundations of Software Testing: ISTQB Certification*. Cengage Learning.

[4] Materi kuliah Software Testing, Sesi 8: Unit Testing (Tools: JUnit), Universitas Katolik Widya Mandala Surabaya.